

Robot Learning: From Fundamentals to Foundation Models

Lecture 8: World Models

Oier Mees

ETH zürich



Microsoft

13.04.2026

Mid-term feedback

- Well-received: main lectures, guest lectures & homeworks
- Things to improve: paper presentations, more time for main lecture & a break, more ECTS credits for workload

What is your favorite aspect of the course so far? (What should we keep doing?)

SPEAKRS and homeworks. As annoying as they are, the homeworks are very cool. I think you managed to ace it in terms of content and length. 🙌🙌🙌🙌🙌 great course. I am someone who doesn't tend to like courses because of their lack of practicality, but this one is lit.

What I get out of the paper presentation: mhm okay this speaks about this. If I like it then I can maybe read it or go through it. Otherwise i forget it. I think it's great, because tomorrow I have a list of papers that you consider important and that in this field everybody knows about. About the presentations themselves, I doubt their usefulness since they are very short and I most of the time don't understand a lot of things

Changes

- We will move one additional paper presentation from Monday's to Thursdays
 - More time for main lecture & a break

Projects: Key Information

- 5 Projects & competitions, **Demo day on May 21th**
 - **Best team for per project will have potential chance to demo to Yann LeCun, Shuran Song, Jitendra Malik & others on May 29th (<https://vreal.ch>)**
- NVIDIA supports the course with 12500 H100 hours, ~277 H100 hours per team

Project 1: Reasoning Pick&Place - VLA

- Eval 1 (50pts): Pick up a plastic banana and place it into one of 3 bowls in front of the arm
- Eval 2 (50pts): Perform the same task, but the language instruction this time requires your VLA to reason about which bowl to put the banana into (i.e. like “put the banana into the 2 minus 1 bowl from the left”)
- Eval 3 (50pts): Place a coke can on a certain image (you won't know the images beforehand)
- Bonus (50pts): We give bonus points for the smallest model in number of parameters



“Move Coke Can near Taylor Swift”

Project 2: Pushing - World Model

- Eval 1 (50pts): Push a given smaller object (TBD which one) in a straight line
- Eval 2 (50pts): Push a given smaller object around an obstacle
- Eval 3 (50pts): Push a new unknown object in a straight line (25pts) and around an obstacle (25pts again)
- Bonus (50pts): Least amount of pushes needed in each of the three challenges (10 pts for least pushes for the first, 10 for the second, 30 for the last, will be divided amongst the 10 groups competing).



Project 3: Singulation - Reinforcement Learning

- Eval 1 (50pts): Learn a pick and place policy that picks up one wooden block and placing them in one of three bowls laid out in front of the arm (can be just BC).
- Eval 2 (50pts): Singulate the combined four wooden blocks
- Eval 3 (50pts): Singulate more combined blocks.
- Bonus (50pts): Fastest singulating policy



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM -> keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawnd on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM -> keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawnd on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM -> keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawned on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM -> keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawned on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM $\rightarrow$ keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawned on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 4: Keyboard Typing - Code as Policies

- Design a system (VLM -> keypoints + some classical keypoint grasping policy) that is able to work in unseen eval environment.
- Eval 1 (50pts): Move to four points drawnd on a paper sheet.
- Eval 2 (50pts): Pressing defined keys on a laptop keyboard.
- Eval 3 (50pts): Typing words on an unseen laptop keyboard.
- Bonus (50pts): Fastest spelling of words on the keyboard.



Project 5: Cloth Folding - Diffusion Policy

- Milestone (50pts): Grab one edge.
- Milestone (50pts): Make one fold.
- Milestone (50pts): Second fold.
- Bonus (50pts): Fold the full cloth & fastest policy.



Robot Learning: From Fundamentals to Foundation Models

Lecture 8: World Models

Oier Mees

ETH zürich



Microsoft

13.04.2026

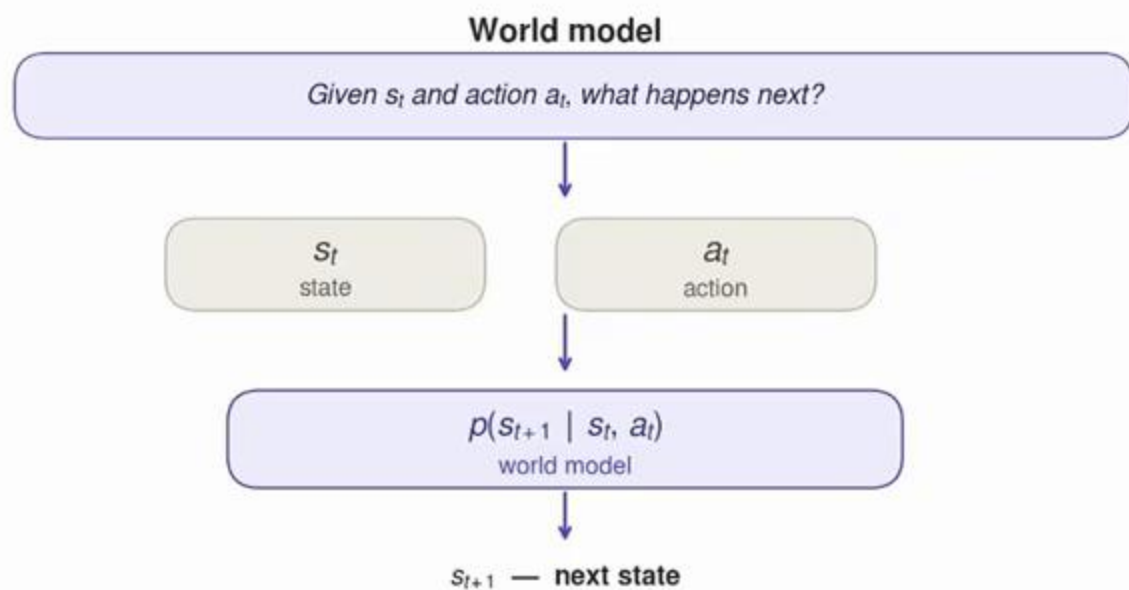
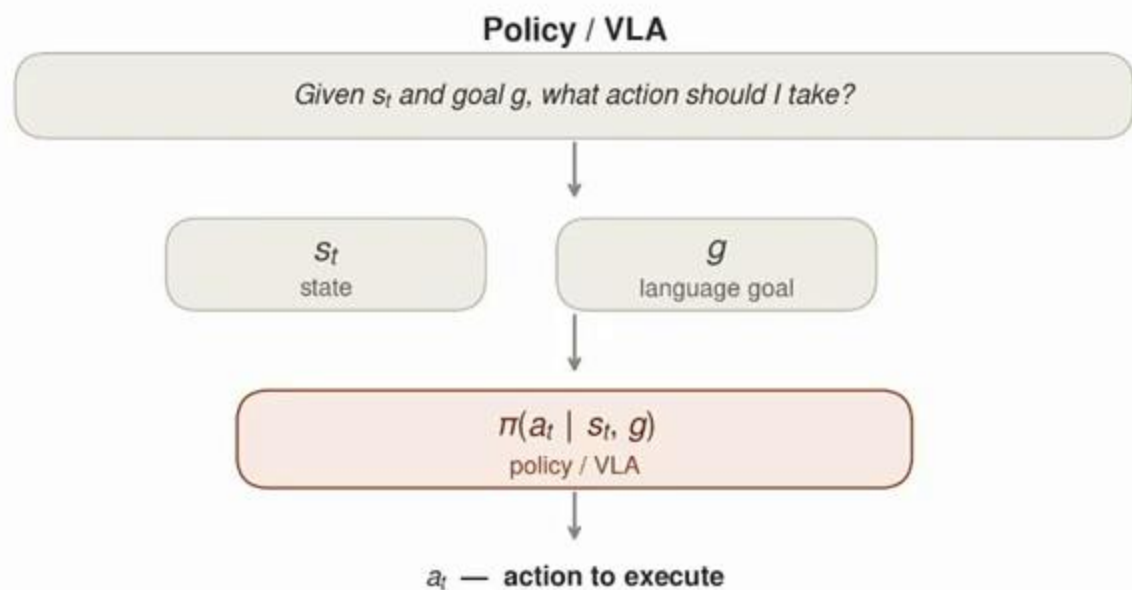
World Models

- Batters can hit 160km/h balls by anticipating the location
- Our world model allows to decide batting in shorter time than visuals reach our brain



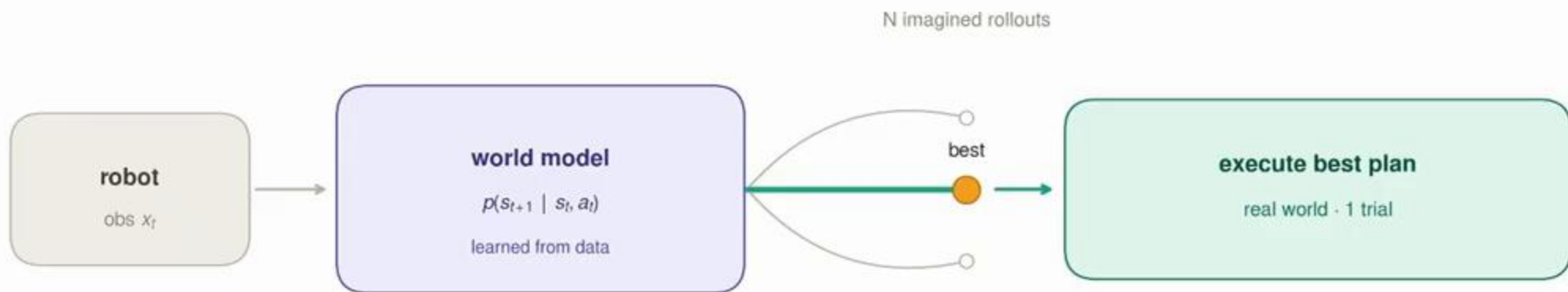
World Models vs Policies

- Policies/VLAs are blind to physical causality & temporal dynamics



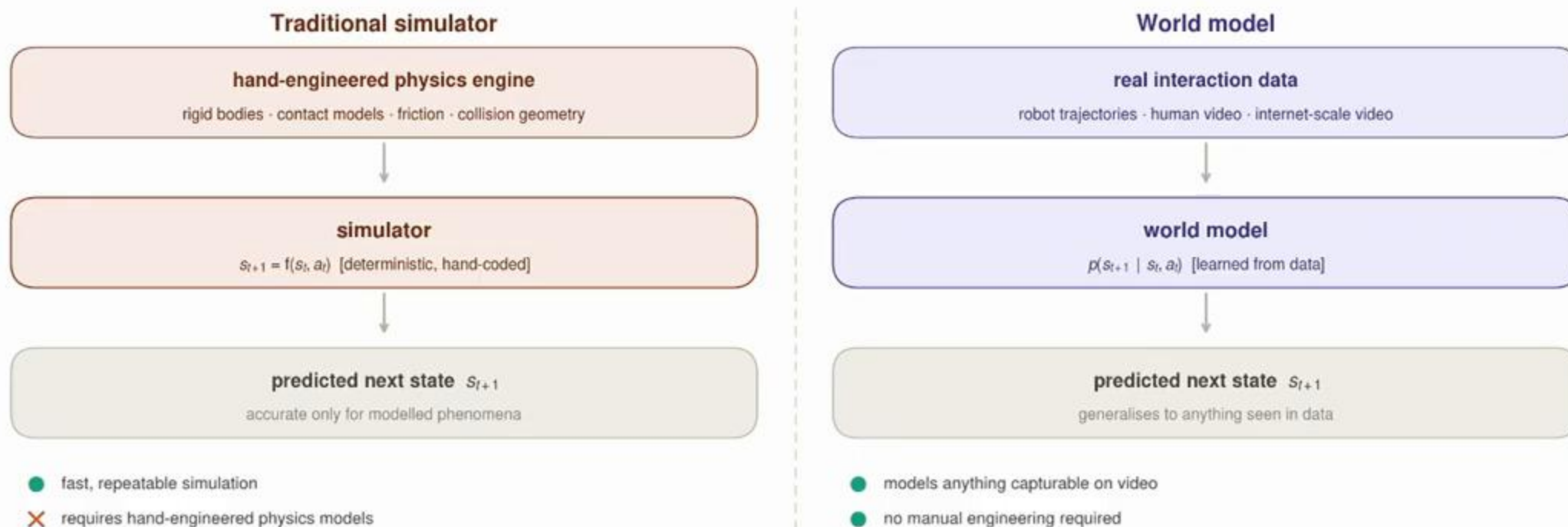
Why World Models?

- What if we could predict how different sequence of actions would affect the environment?
 - Unlock optimizing for the optimal trajectory



World Models: Data-driven Simulators

- Function that predicts the consequences of actions, i.e. a learned simulator



World Model Definition

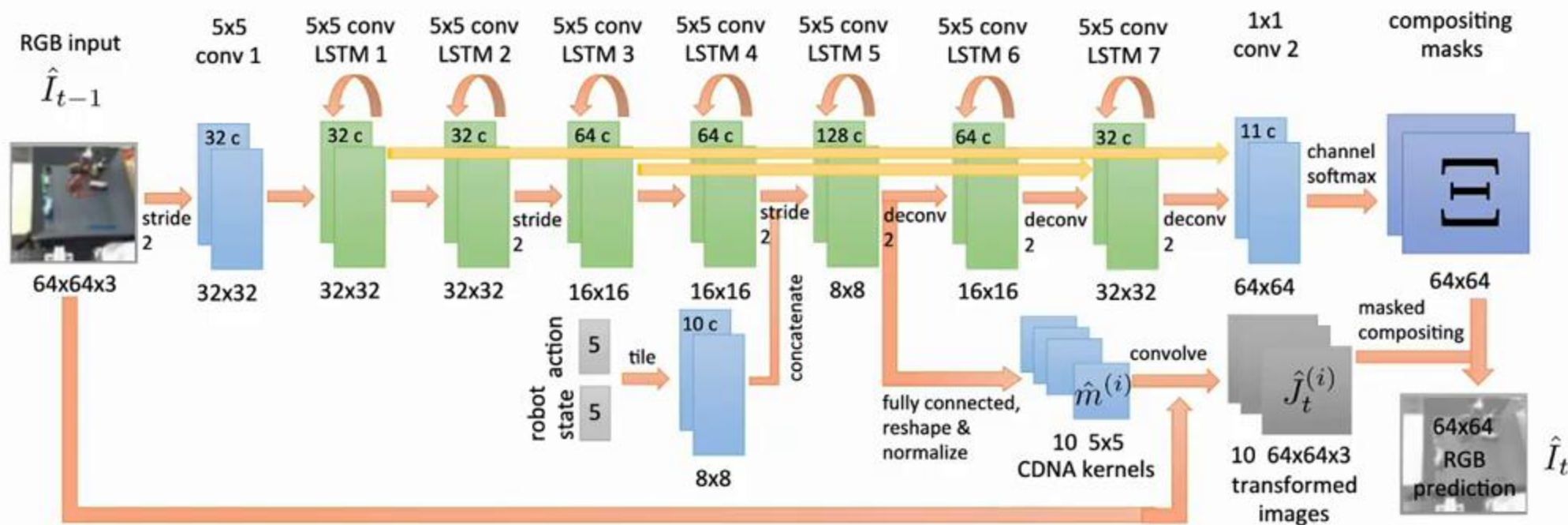
- Given the current state, an action, and a memory of the past — what happens next?

$$P(s_{t+1} \mid s_t, a_t, h_t)$$

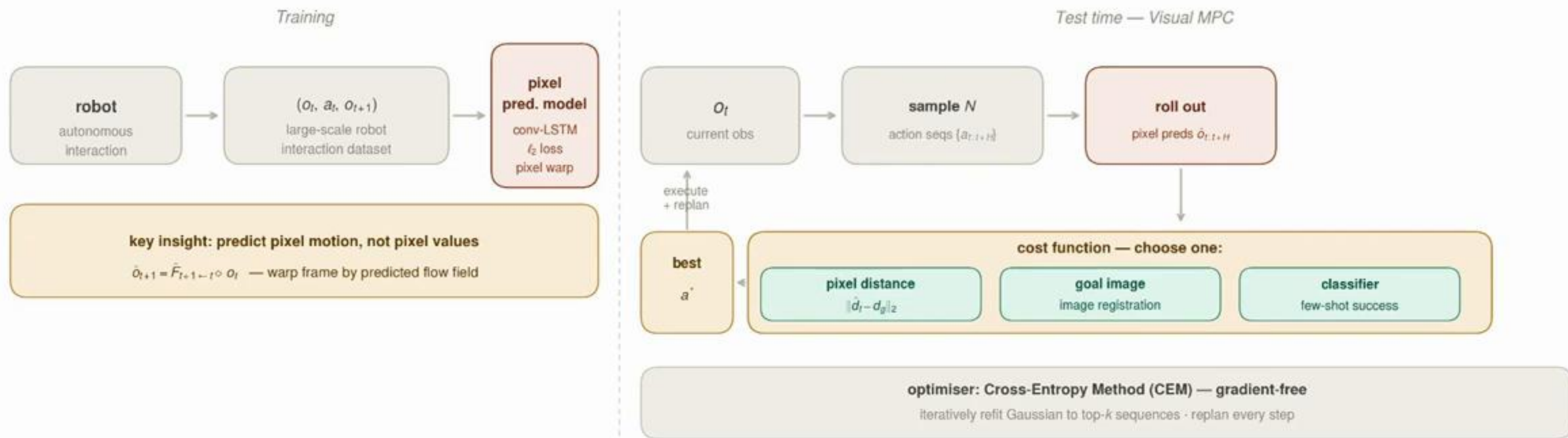
- Strictly, *video models* $P(\text{video} \mid \text{text})$ *are not world models*

Pixel AC-WMs

- Instead of full video prediction, predict action conditioned flow field that warps $\hat{F}_{t+1 \leftarrow t}$ current image to next frame



Pixel AC-WMs



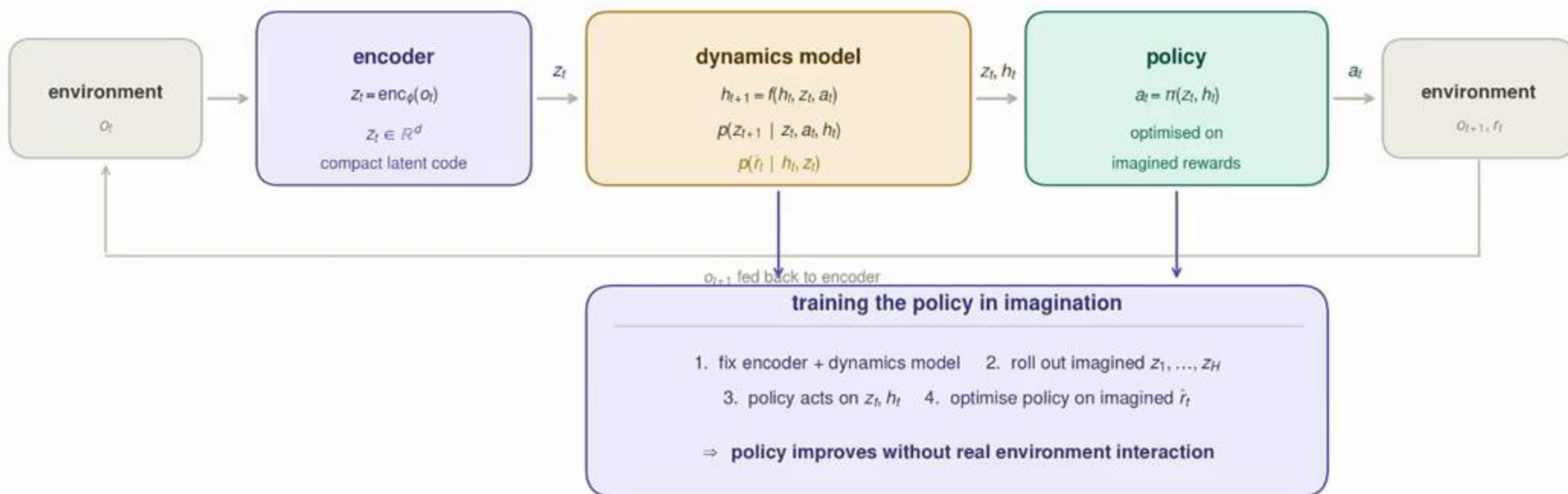
- ✓ Self-supervised training on robot videos
- ✓ Generalizes to unseen objects due to motion objective
- ✗ Blurry predictions with L2 loss
- ✗ Planning in pixel space computationally expensive
- ✗ Prediction error accumulates

What if we compressed observations into a latent space and trained a policy entirely inside that learned world?

Latent AC-WMs

- By learning a compact, latent world model a policy can be trained entirely in imagination

Latent AC-WMs — general structure



The OG Latent AC-WM

At each time step, our agent receives an **observation** from the environment.

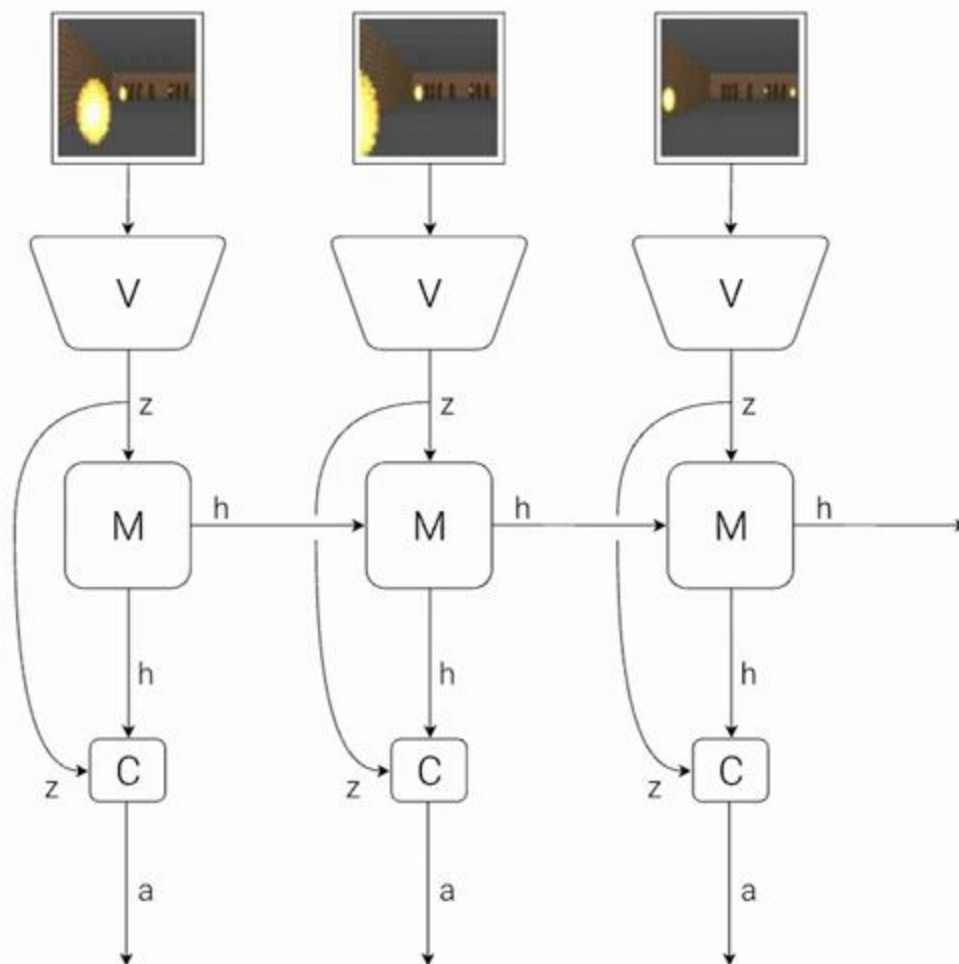
World Model

The **Vision Model (V)** encodes the high-dimensional observation into a low-dimensional latent vector.

The **Memory RNN (M)** integrates the historical codes to create a representation that can predict future states.

A small **Controller (C)** uses the representations from both **V** and **M** to select good actions.

The agent performs **actions** that go back and affect the environment.

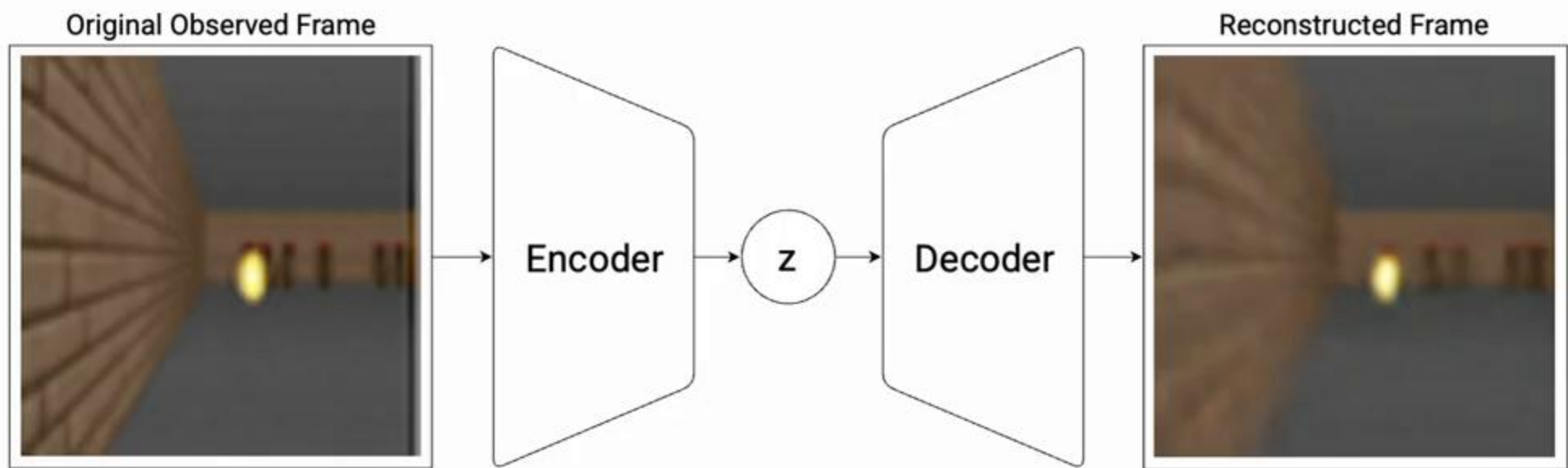


World Models

Ha and Schmidhuber (2018)

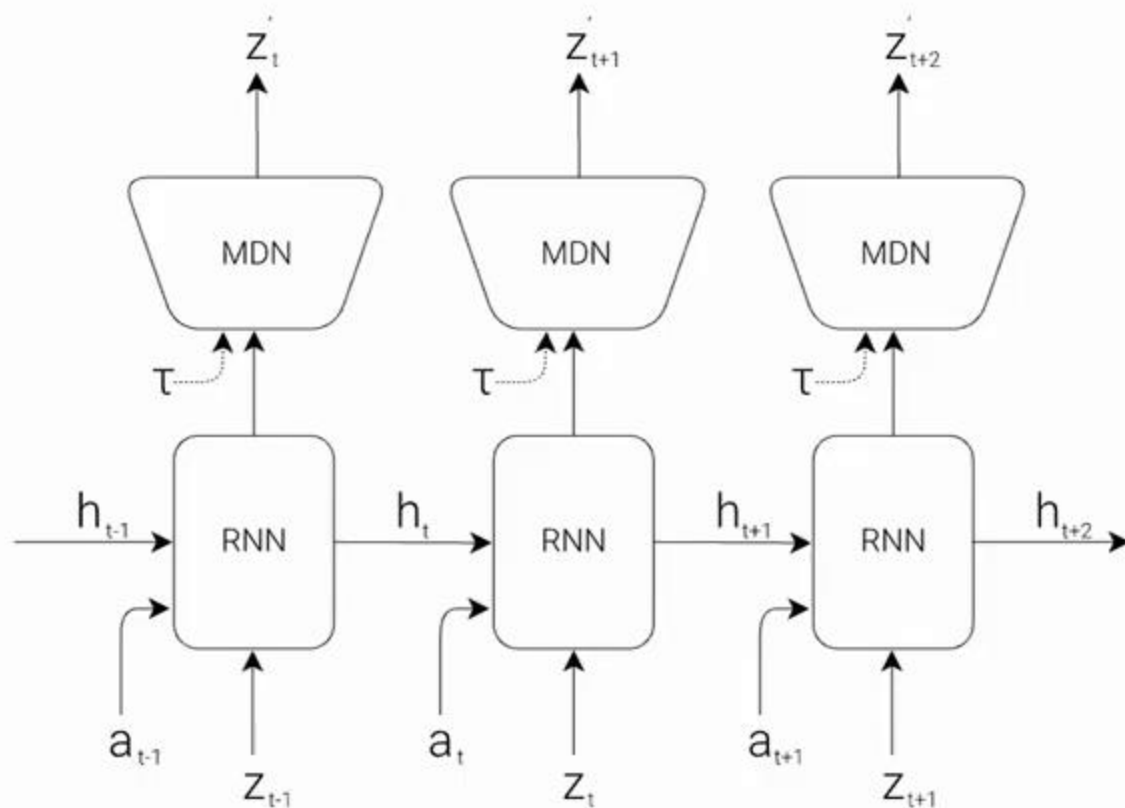
The OG Latent AC-WM: Visual Model

- Learn abstract, compressed representation of each input frame



The OG Latent AC-WM: Memory Model

- Compress what happens over time $P(z_{t+1} | a_t, z_t, h_t)$
 - The MDN output models $P(z_{t+1})$ as a mixture of Gaussians



World Models

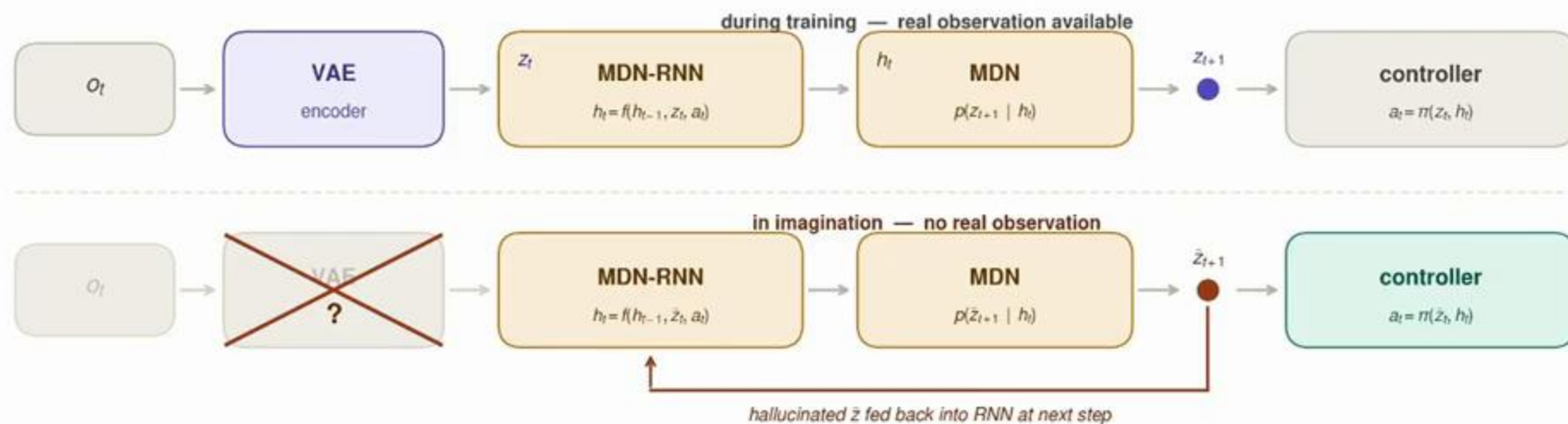
Ha and Schmidhuber (2018)

The OG Latent AC-WM: Controller Model

- Policy conditions on both the current latent z_t and the memory h_t , single FC layer $a_t = \pi(z_t, h_t)$
 - Trained with black-box evolutionary optimisation algorithm (CMA-ES)
 1. Sample N controllers from $W_c(i) \sim \mathcal{N}(\mu, \Sigma)$
 2. For each candidate run dream rollout and collect imagined reward
 3. Select top rollouts
 4. Update μ and Σ toward the top rollouts

Why Imagination Without Priors Drifts

- RNN is trained on real z_t from VAE, not own predictions
- No principled way to generate z_t from h_t alone, $P(z_t | h_t)$



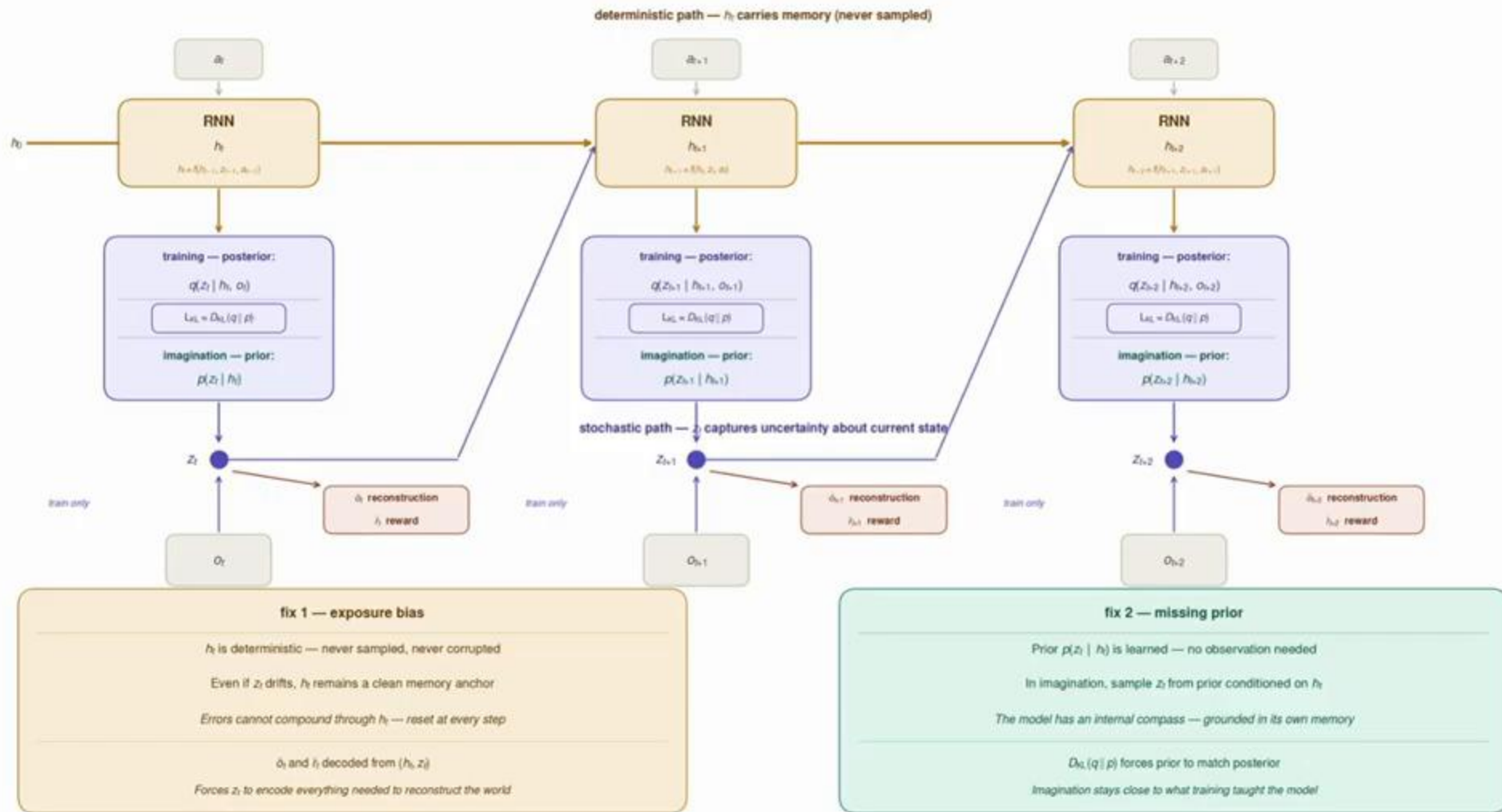
problem

VAE is trained separately from the RNN — it encodes o_t , not the model's own belief

There is no prior $p(z_t | h_t)$ — without o_t the model has no way to form a belief

Sampling errors in $\bar{z}_t$ compound across steps $\Rightarrow$ imagination drifts from reality

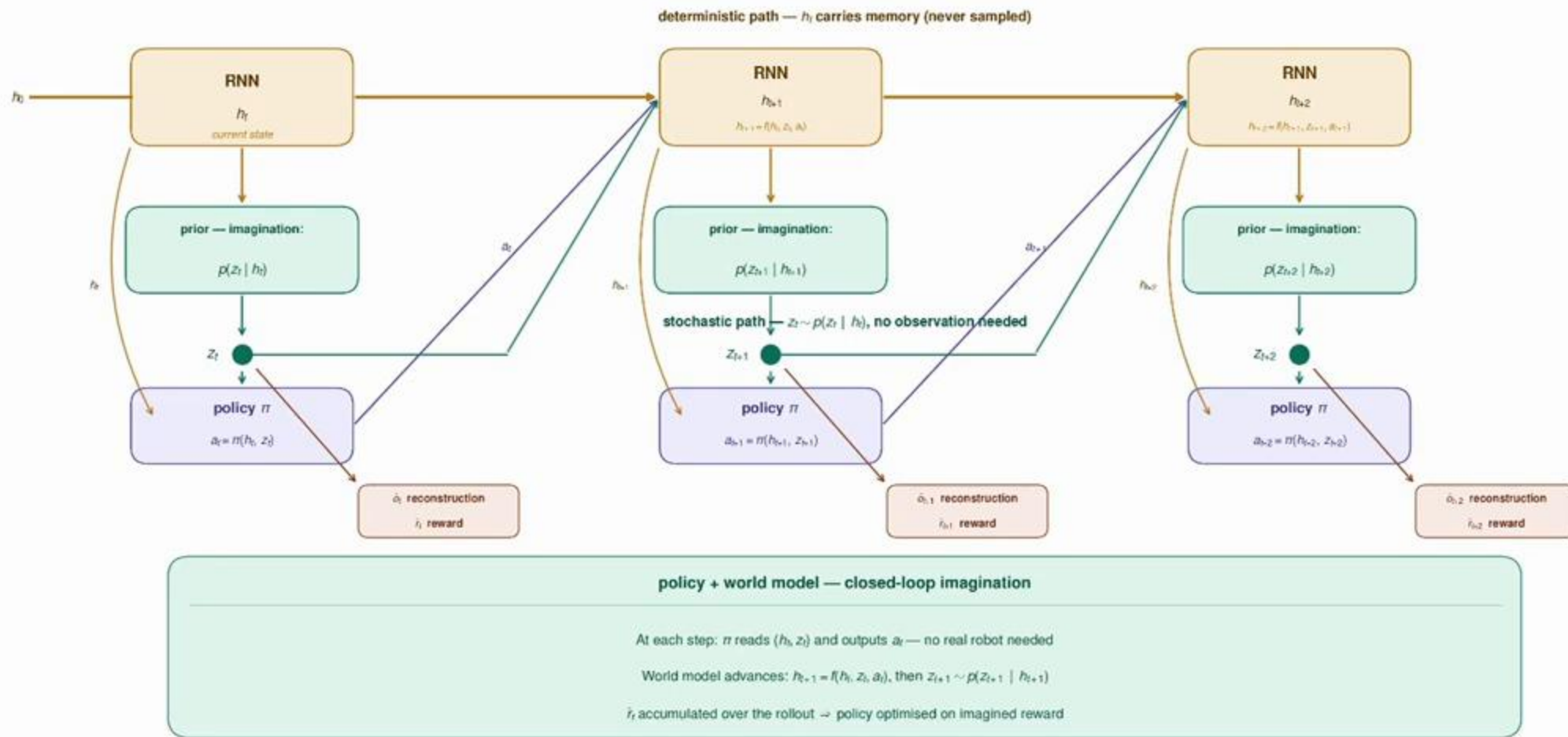
Training a Recurrent State-Space Model (RSSM)



Learning Latent Dynamics for Planning from Pixels

Hafner et al. (2019)

RSSM Closed-Loop Inference Imagination

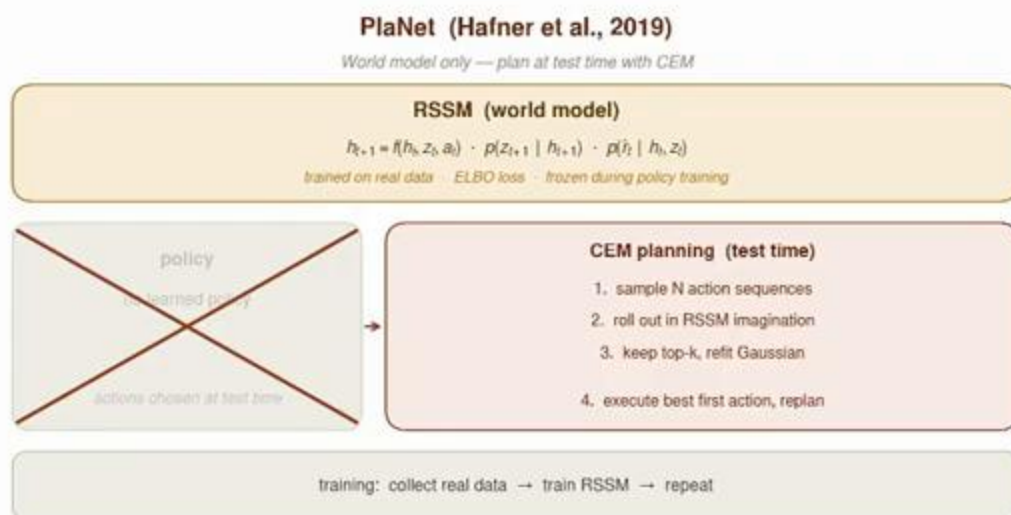


Learning Latent Dynamics for Planning from Pixels

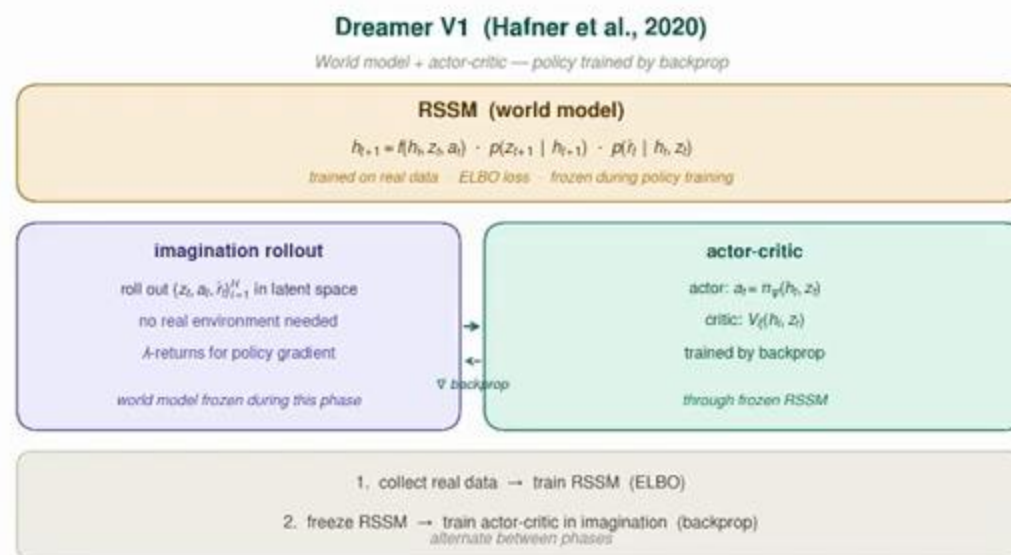
Hafner et al. (2019)

Dreamer V1

- Why plan from scratch with CEM at every step when you could amortise that planning into a learned policy?

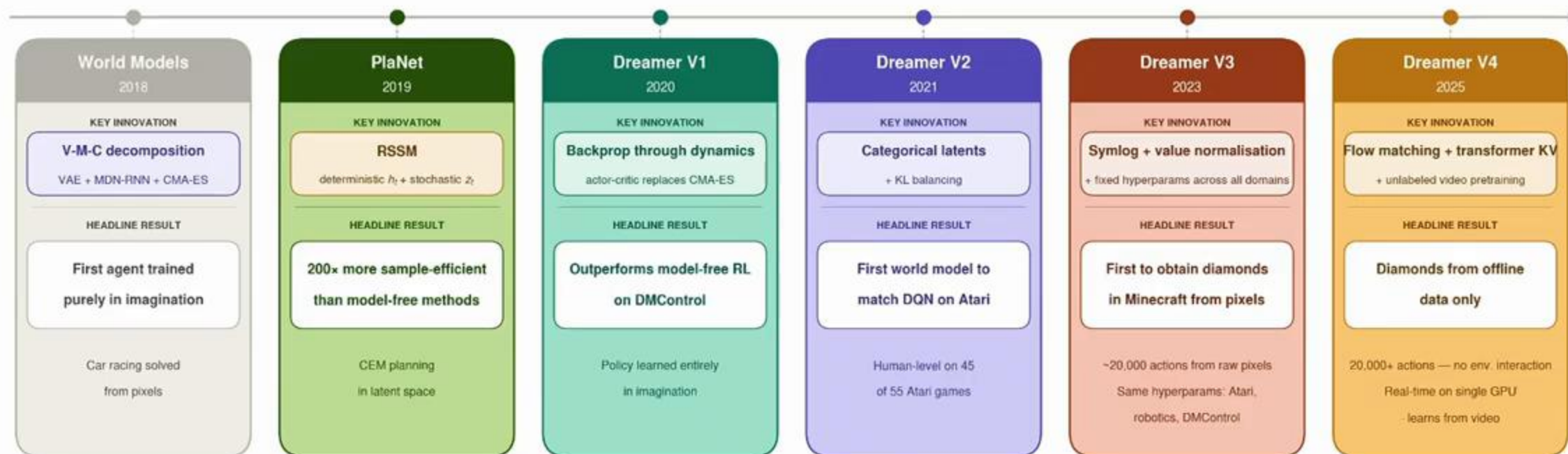


- ✓ Actions chosen without policy bias
- ✗ Planning from scratch at every step CEM



- ✓ Backprop through differentiable dynamics enables long-horizon planning
- ✗ Policy can exploit world model errors

Dreamer Lineage



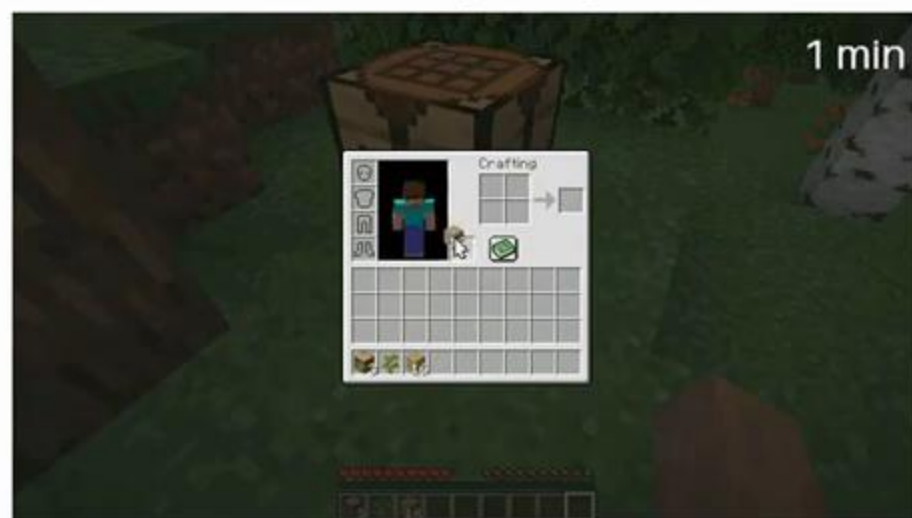
Dreamer V4: Decoupling Videos & Actions

- Pretrain dynamics model on *unlabeled videos* via shortcut forcing/flow matching $p(z_{t+1} | z_{\leq t})$, finetune $p(z_{t+1} | z_{\leq t}, a_{\leq t})$



Memory: Replace RNN hidden states with Transformer KV cache

Learns Minecraft Diamonds,
20000 decisions, **purely offline**



Works on offline dataset from my Soar paper



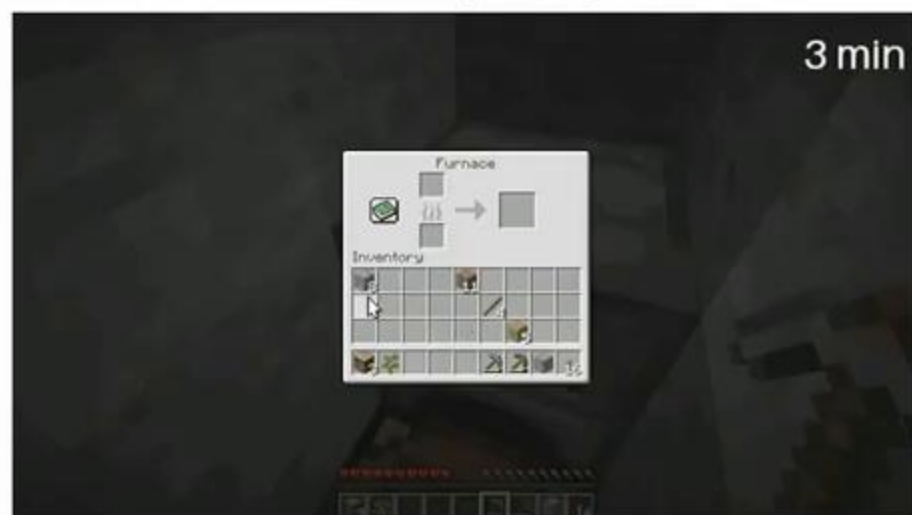
Dreamer V4: Decoupling Videos & Actions

- Pretrain dynamics model on *unlabeled videos* via shortcut forcing/flow matching $p(z_{t+1} | z_{\leq t})$, finetune $p(z_{t+1} | z_{\leq t}, a_{\leq t})$



Memory: Replace RNN hidden states with Transformer KV cache

Learns Minecraft Diamonds,
20000 decisions, **purely offline**



Works on offline dataset from my Soar paper



Dreamer's latents are domain specific. What if we want to scale the videos to all of Internet?



Generative Video Models

Naïve Video Tokenization

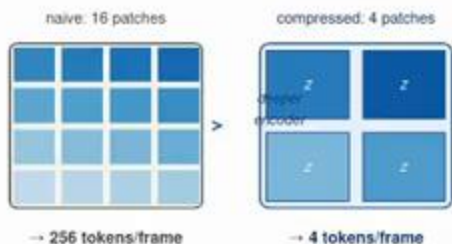
- Per-frame ViT produces too many tokens



Video Tokenization Compression

Axis 1 — Spatial Compression

deeper encoder → fewer tokens per frame



mechanism

VQ-VAE / VQGAN encoder with stride
8× spatial: $256 \times 256 \rightarrow 4 \times 4$ tokens

examples

VQGAN · SD-VAE · Cosmos CV8x8x8

causal? yes — no temporal dependency

Axis 2 — Temporal Compression

merge frames → fewer time steps



Tubelets

3D patch (t,h,w)
full tube at once

non-causal

TimeStorner · ViViT

Cosmos (analysis)

Causal Aggregation

attend past only
frame-by-frame decode

causal

Dreamer V4 tokenizer

Cosmos world model

Axis 3 — Adaptive Compression

variable budget → complex frames get more tokens



token budget allocated proportionally to frame complexity

mechanism

tail-drop masking during training
tokens ordered by information content

examples

ElasticTok (Yan et al., ICLR 2025)

causal? yes — works per-frame with context

256 → 4 tokens / frame

64× spatial reduction

200 → 25 time steps

8× temporal reduction

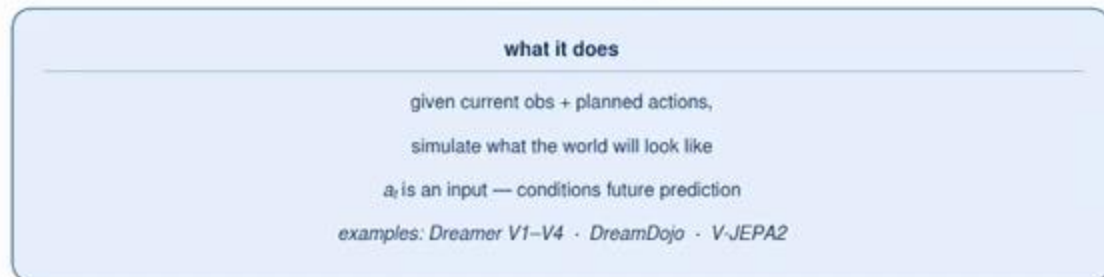
variable tokens per frame

2–5× reduction on average

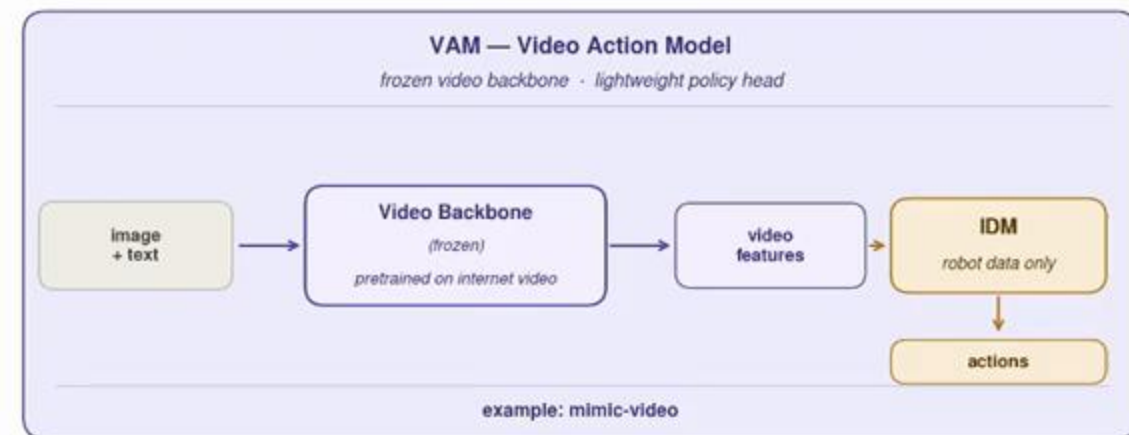
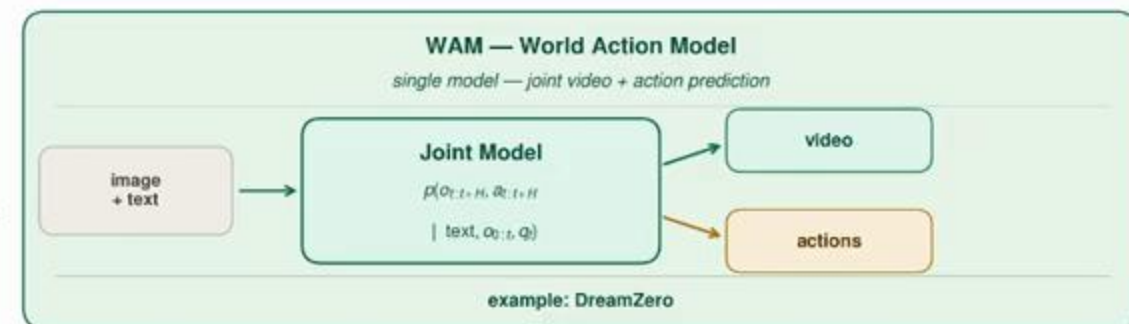
Combined (Cosmos): spatial 8× + temporal 8× = 512× compression → 51,200 tokens becomes 100 tokens for a 10-sec clip

Axes are independent and composable · causality requirement determines which temporal method to use

Video World Model: Where Do Actions Live?



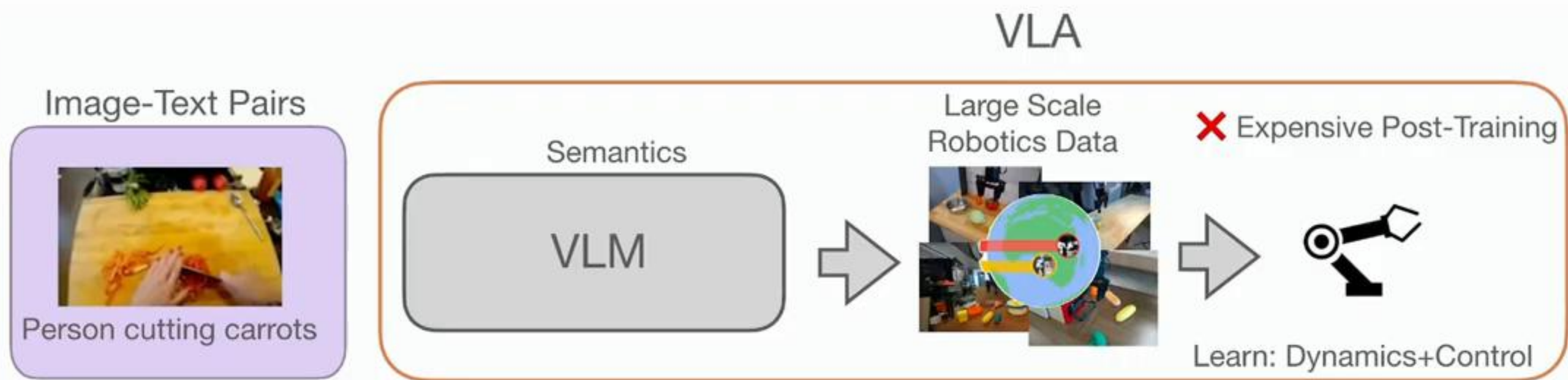
VS



Didn't we say video is expensive?
Why use video backbones?

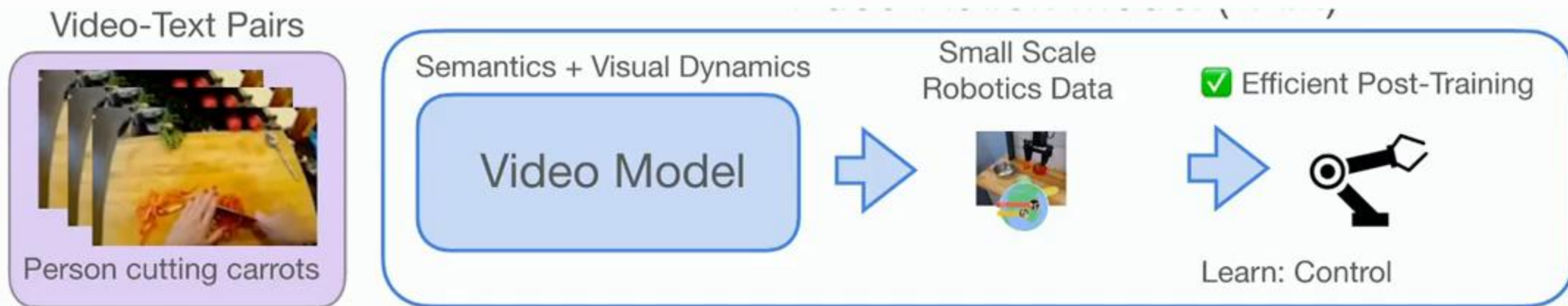
VLA: Poor Sample Efficiency

- VLMs are often trained on static images & text, blind to physical causality and temporal dynamics
 - Large-scale teleop data necessary to convert a VLM to VLA



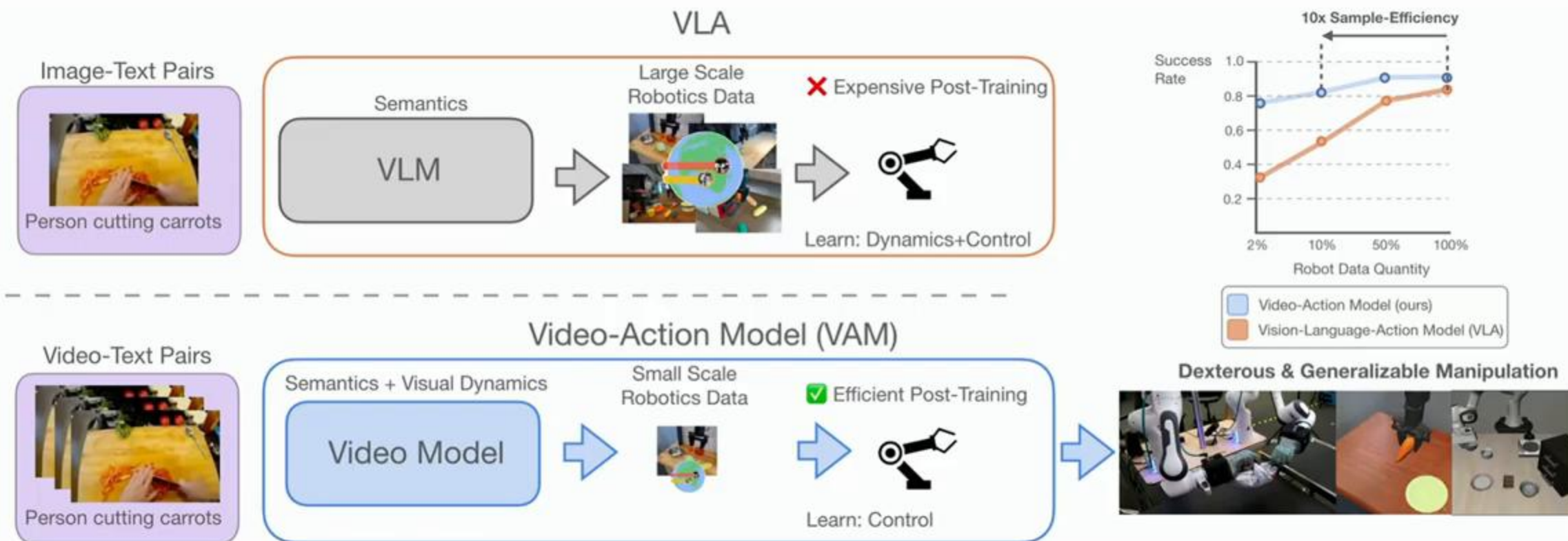
Video Backbones

- Offload dynamics learning to scalable action-free video data



Video-Action Models

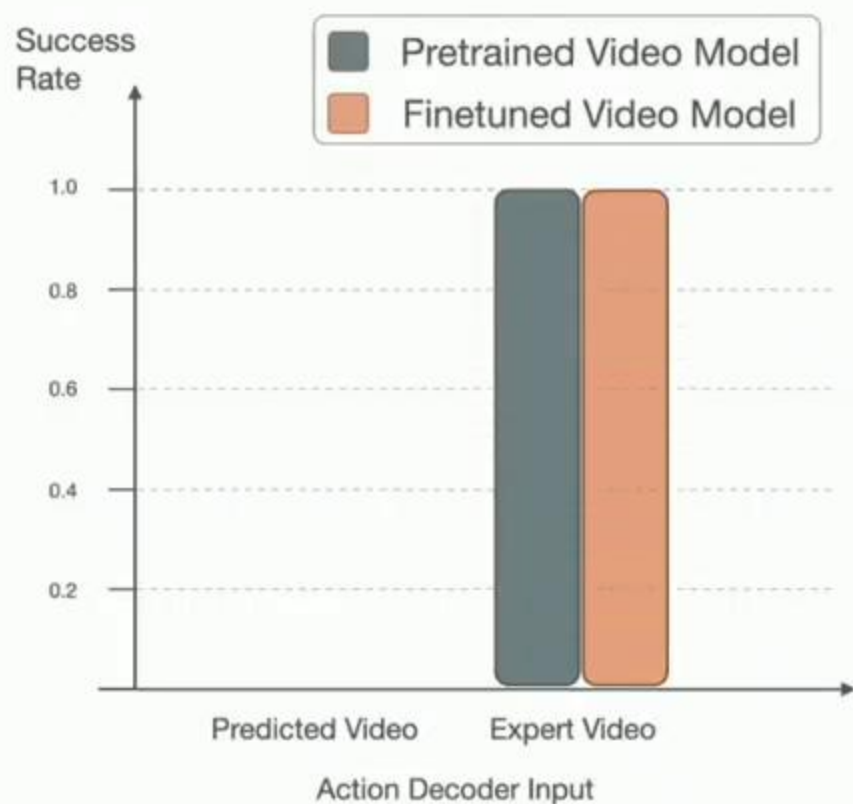
- Key: leverage **generative video models** as backbone



mimic-video: Video-Action Models for Generalizable Robot Control Beyond VLAs

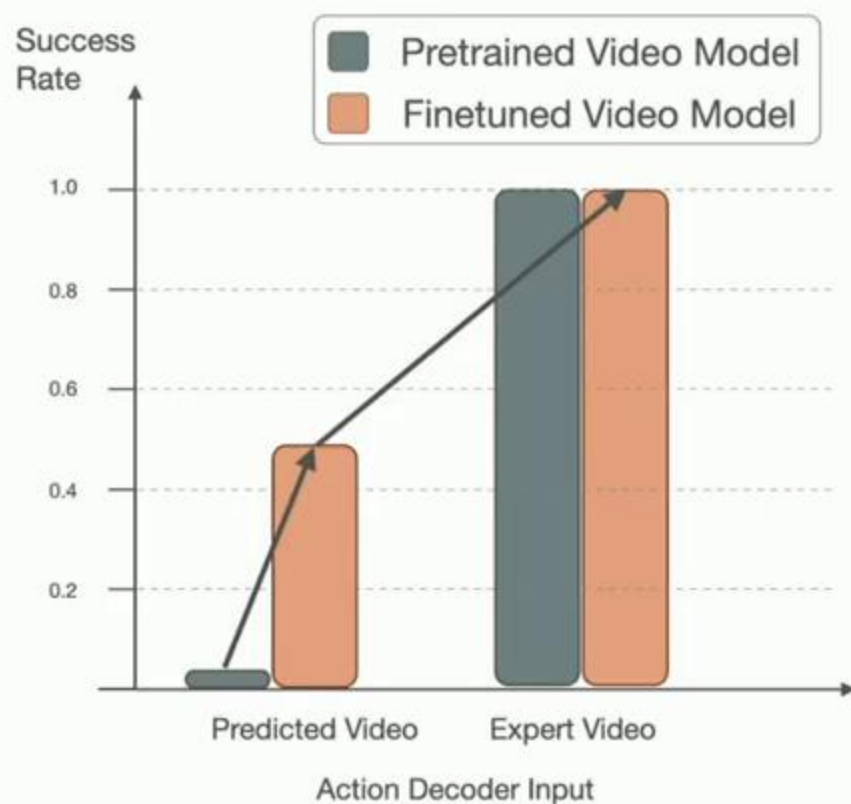
Pai*, Achenbach*, Montesinos, Forrai, Mees*, Nava*. arxiv, 2025.

Policy Performance Scales with Video Model



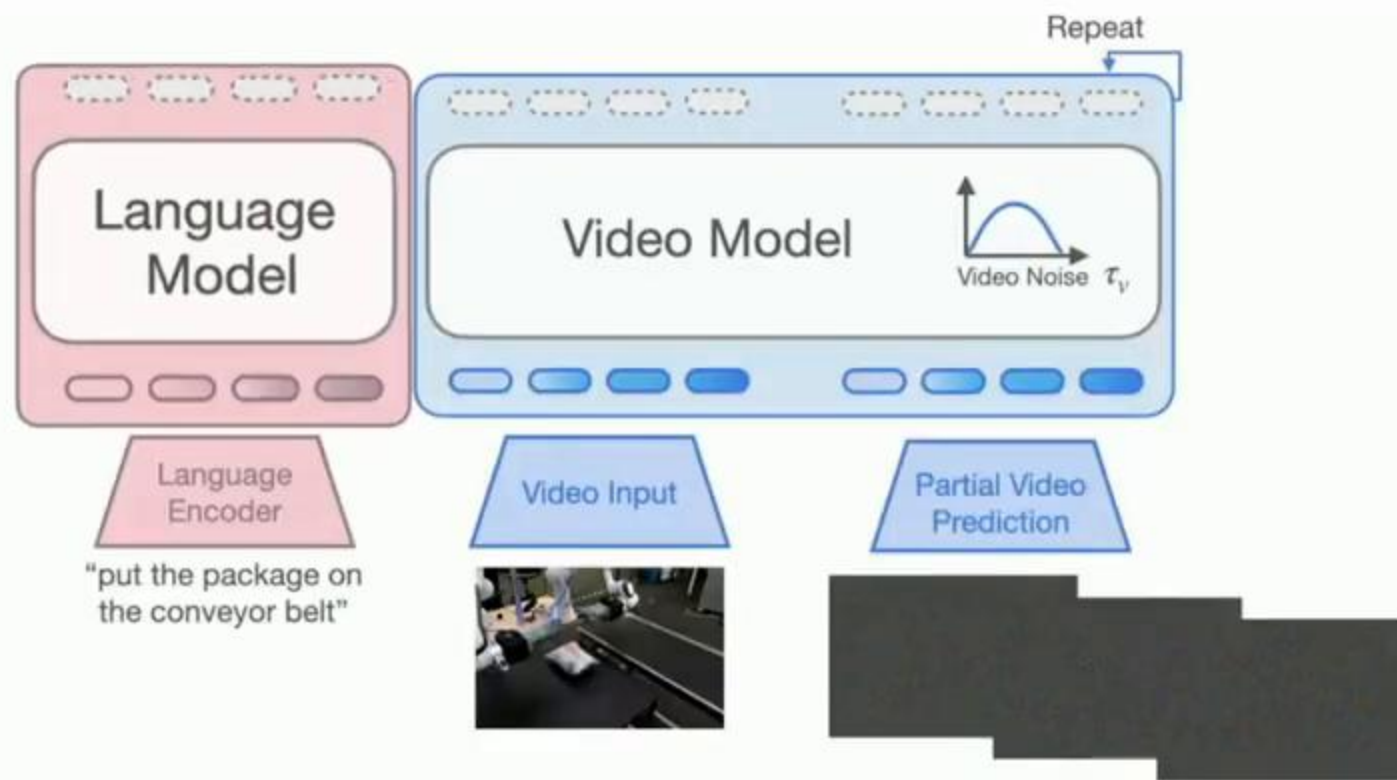
✓ Policy achieves perfect performance when conditioned on ground truth video

Policy Performance Scales with Video Model



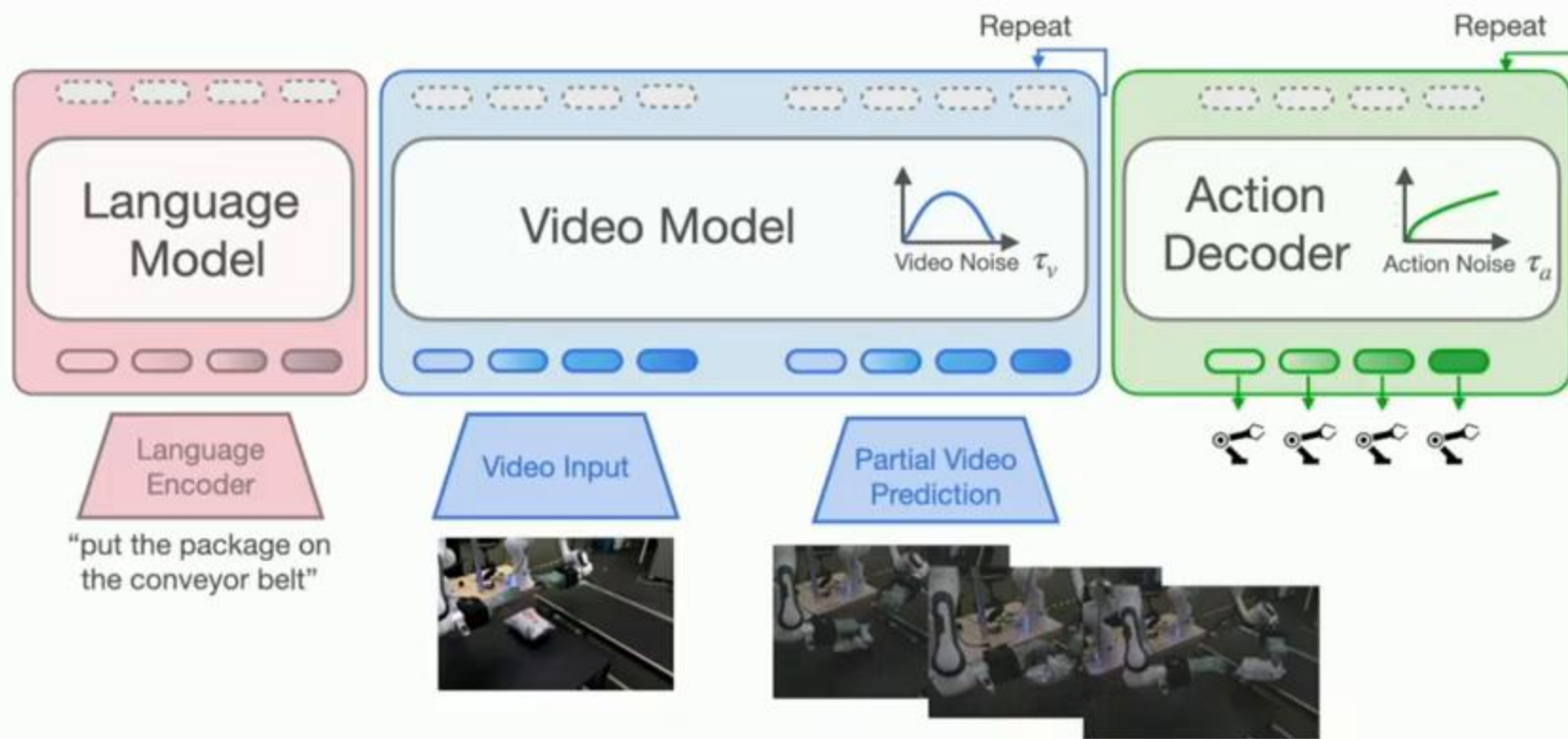
- ✓ Policy achieves perfect performance when conditioned on ground truth video
- ✓ Finetuning a video model on robot data improves video prediction quality and thus policy performance
- ✓ Policy performance scales with better video models

mimic-video



mimic-video first partially denoises future video via flow matching

mimic-video



Then, video model activations condition a light-weight flow matching action decoder

Joint Video-Action Sampling

Autonomous Policy Execution

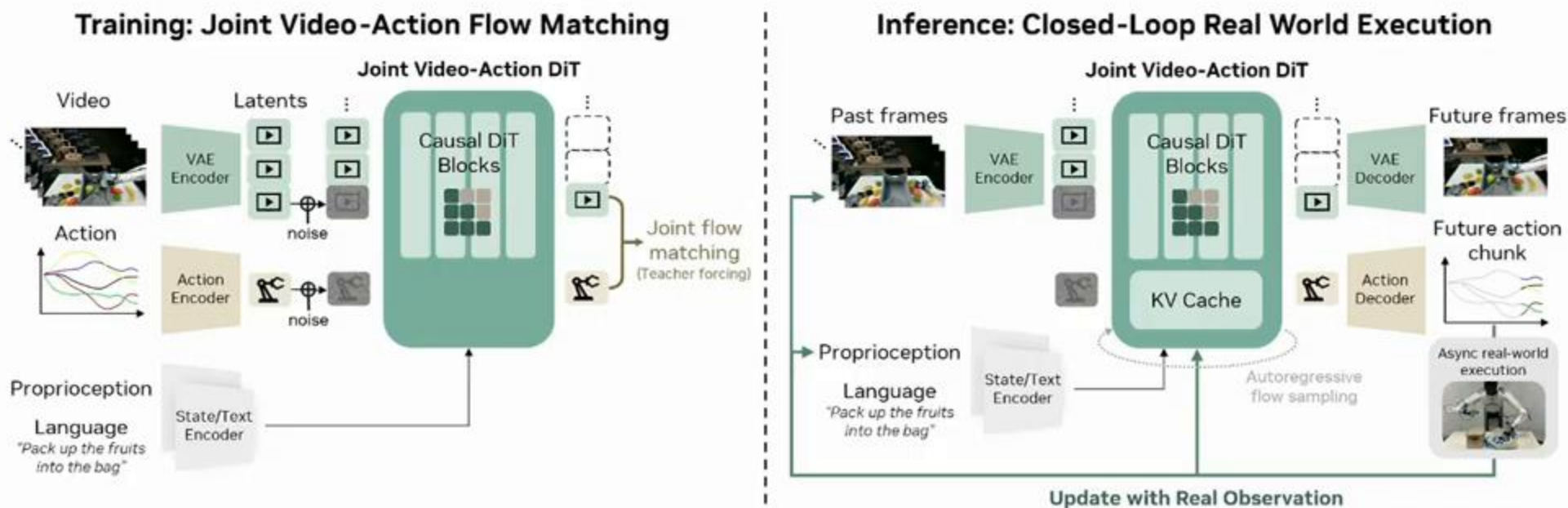
Video Generation

(skipped in regular policy inference)



World Action Models: DreamZero

- Joint video-action prediction, no explicit IDM
- Autoregressive video chunk generation, faster KV cache inference with 14B model



World Action Models are Zero-shot Policies

Ye et al. arxiv, 2026.

WAMs vs VAMs

Dimension	WAMs (DreamZero, mimic-video)	AC-WMs (Dreamer, DreamDojo)
Data	 instruction-labeled only — hard to use play data or failure trajectories at scale	 all robot data: play, failures, policy rollouts — easier to scale
Cross-embodiment	 easy — inputs/outputs embodiment-agnostic only action decoder needs robot data	 open challenge — each robot has its own action space
Pre-training preservation	 small gap from pre-trained video model preserves general visual capabilities	 action conditioning changes input distribution may destroy pre-trained abilities
Beyond imitation	 BC paradigm only — instructions $\rightarrow$ actions no RL or counterfactual simulation	 RL in imagination + fine-grained gradient-based planning
Planning	 best-of-N via diverse text prompts easy action proposal generation	 fine-grained gradient-based optimization of action sequences at inference time

World Action Models: DreamZero

- Strong generalization and few-shot adaptation capabilities



Pick up the red apple on the table and place it on the gray towel. I 🟢



Fold the bottom of the green short sleeve to the middle. Then, pull the shirt toward the edge of the table. Next, fold the top of the shirt down to the middle. Finally, grasp the collar and folds it down. I 🟡



Untie the knot of the shoelace. I 🟢



Pick up the marker and draw a circle on the book. I 🟡

World Action Models are Zero-shot Policies

Ye et al. arxiv, 2026.

WAMs vs VAMs

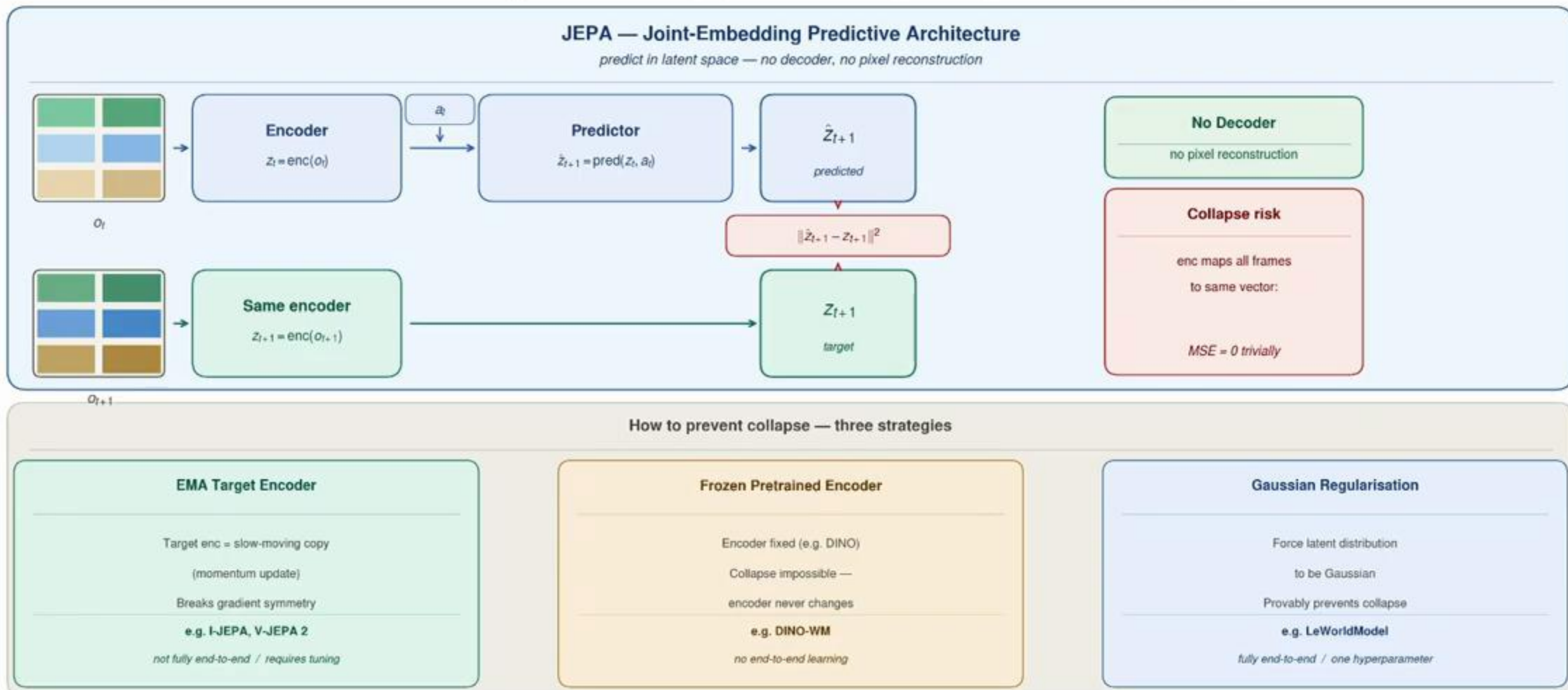
Dimension	WAMs (DreamZero, mimic-video)	AC-WMs (Dreamer, DreamDojo)
Data	 instruction-labeled only — hard to use play data or failure trajectories at scale	 all robot data: play, failures, policy rollouts — easier to scale
Cross-embodiment	 easy — inputs/outputs embodiment-agnostic only action decoder needs robot data	 open challenge — each robot has its own action space
Pre-training preservation	 small gap from pre-trained video model preserves general visual capabilities	 action conditioning changes input distribution may destroy pre-trained abilities
Beyond imitation	 BC paradigm only — instructions $\rightarrow$ actions no RL or counterfactual simulation	 RL in imagination + fine-grained gradient-based planning
Planning	 best-of-N via diverse text prompts easy action proposal generation	 fine-grained gradient-based optimization of action sequences at inference time

All approaches so far had a decoder to reconstruct or generate pixels at some point

Do you even need to predict pixels at all?

Joint-Embedding Predictive Architecture (JEPA)

- What if we don't predict pixels or tokens at all?



Conclusion

One master formula $P(s_{t+1} | s_t, a_t, h_t)$, five different approaches

Family	s_t (state repr.)	a_t role	Planning	Data	Example
Pixel AC-WMs	pixels o_t	input — conditions future frame	visual MPC (CEM)	robot data + action labels	Finn 2016 Visual Foresight
Latent AC-WMs	compact latent z_t	input — conditions latent dynamics	RL in imagination Dreamer-style	robot data + action labels	Dreamer V1-V4 DayDreamer
WAMs	video tokens (full frames)	output — jointly predicted	generate video + extract actions	internet video + robot fine-tune	DreamZero
VAMs	video tokens (frozen backbone)	output — IDM reads from video	generate latent video + IDM	internet video (backbone) + robot data (IDM)	mimic-video
JEPA	latent z_t (no decoder ever)	input — conditions latent predictor	latent MPC no pixels needed	robot data + action labels	LeWorldModel V-JEPA 2

Open questions

Will flexible conditioning win?

one model on text OR actions — best of both worlds

Pixels or latents?

does pixel prediction help cross-embodiment, or is it unnecessary cost?

Can JEPA scale?

V-JEPA 2 shows promise — can latent WMs match generative ones at scale?

Thank you for your attention